

Operating Systems

Dr. Mostafa Elgendy

Computer Science Department,

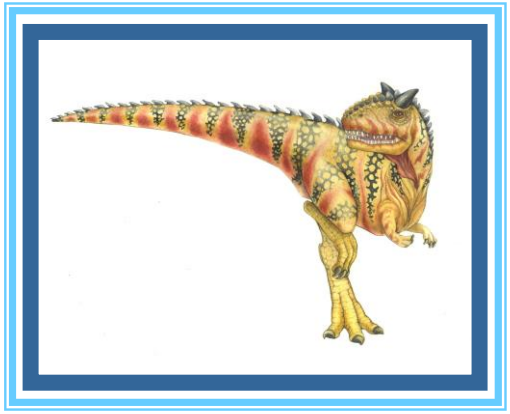
Faculty of Computers and Artificial Intelligence

Benha University

2022



Chapter 4
Threads & Concurrency



Silberschatz, Galvin and Gagne ©2018



Outline

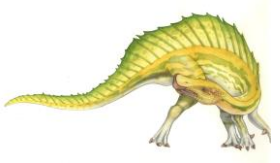
- Overview
- Multicore Programming
- Multithreading Models
- Thread Libraries





Motivation

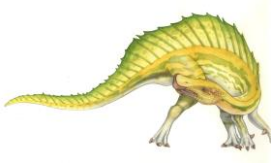
- Most modern applications are multithreaded
- Threads run within application
- Multiple tasks with the application can be implemented by separate threads
  - Update display
  - Fetch data
  - Spell checking
  - Answer a network request





Motivation

- Process creation is heavy-weight while thread creation is light-weight
- Can simplify code, increase efficiency.
- Kernels are generally multithreaded.



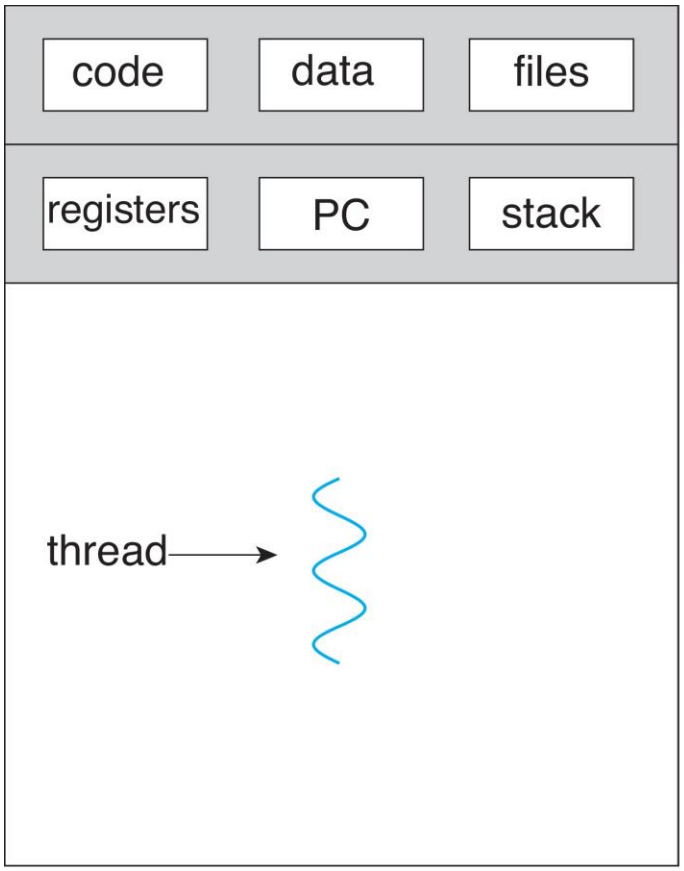


Multicore Programming

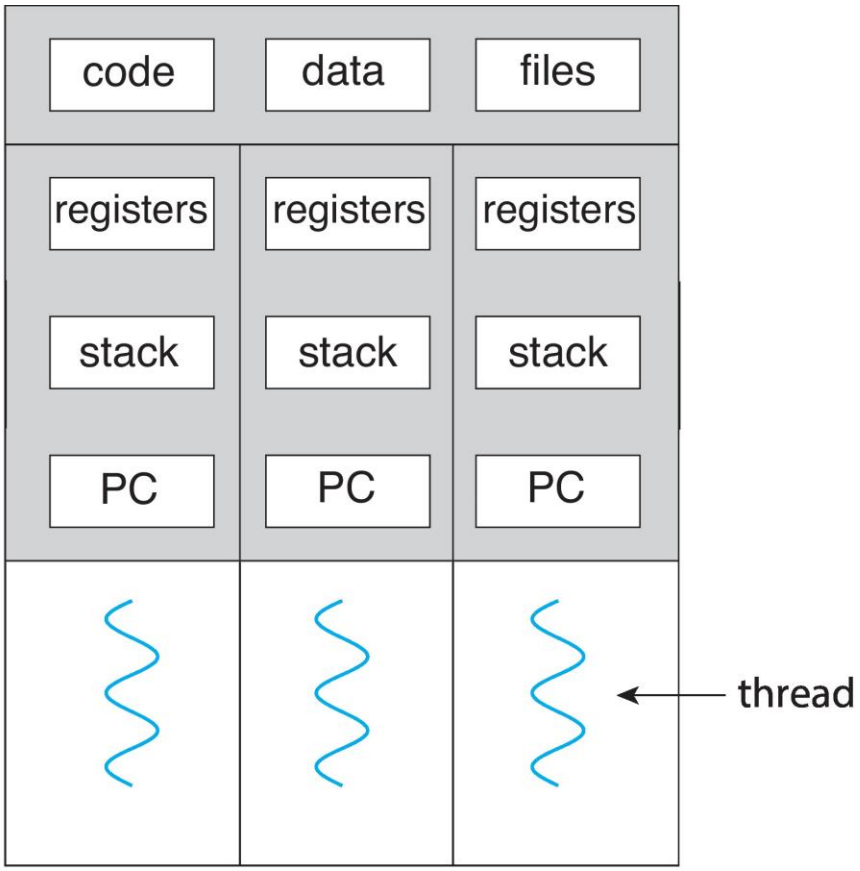




Single and Multithreaded Processes



single-threaded process

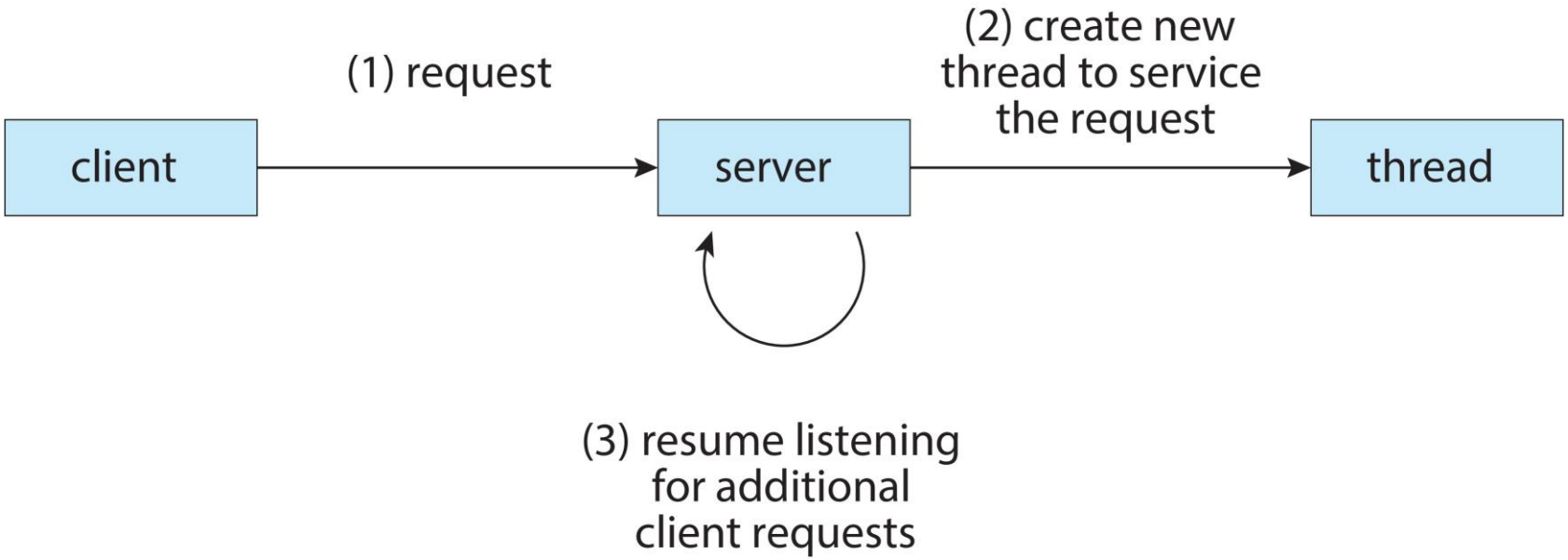


multithreaded process





Multithreaded Server Architecture







Benefits

- **Responsiveness:** may allow continued execution if part of process is blocked, especially important for user interfaces
- **Resource Sharing:** threads share resources of process, easier than shared memory or message passing
- **Economy:** cheaper than process creation, thread switching lower overhead than context switching
- **Scalability:** process can take advantage of multicore architectures





Multicore Programming

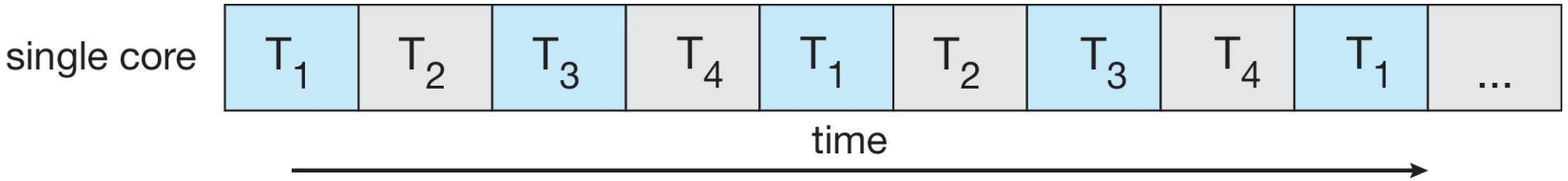
- **Multicore** or **multiprocessor** systems putting pressure on programmers, challenges include:
  - **Dividing activities**
  - **Balance**
  - **Data splitting**
  - **Data dependency**
  - **Testing and debugging**
- **Parallelism** implies a system can perform more than one task simultaneously
- **Concurrency** supports more than one task making progress



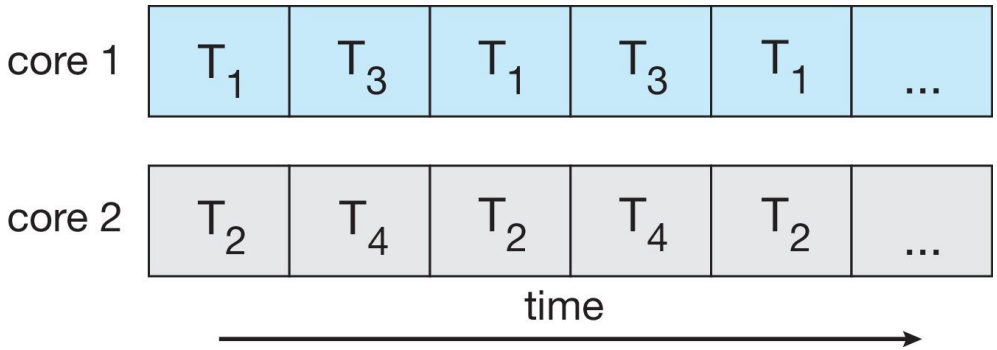


Concurrency vs. Parallelism

- Concurrent execution on single-core system:



- Parallelism on a multi-core system:







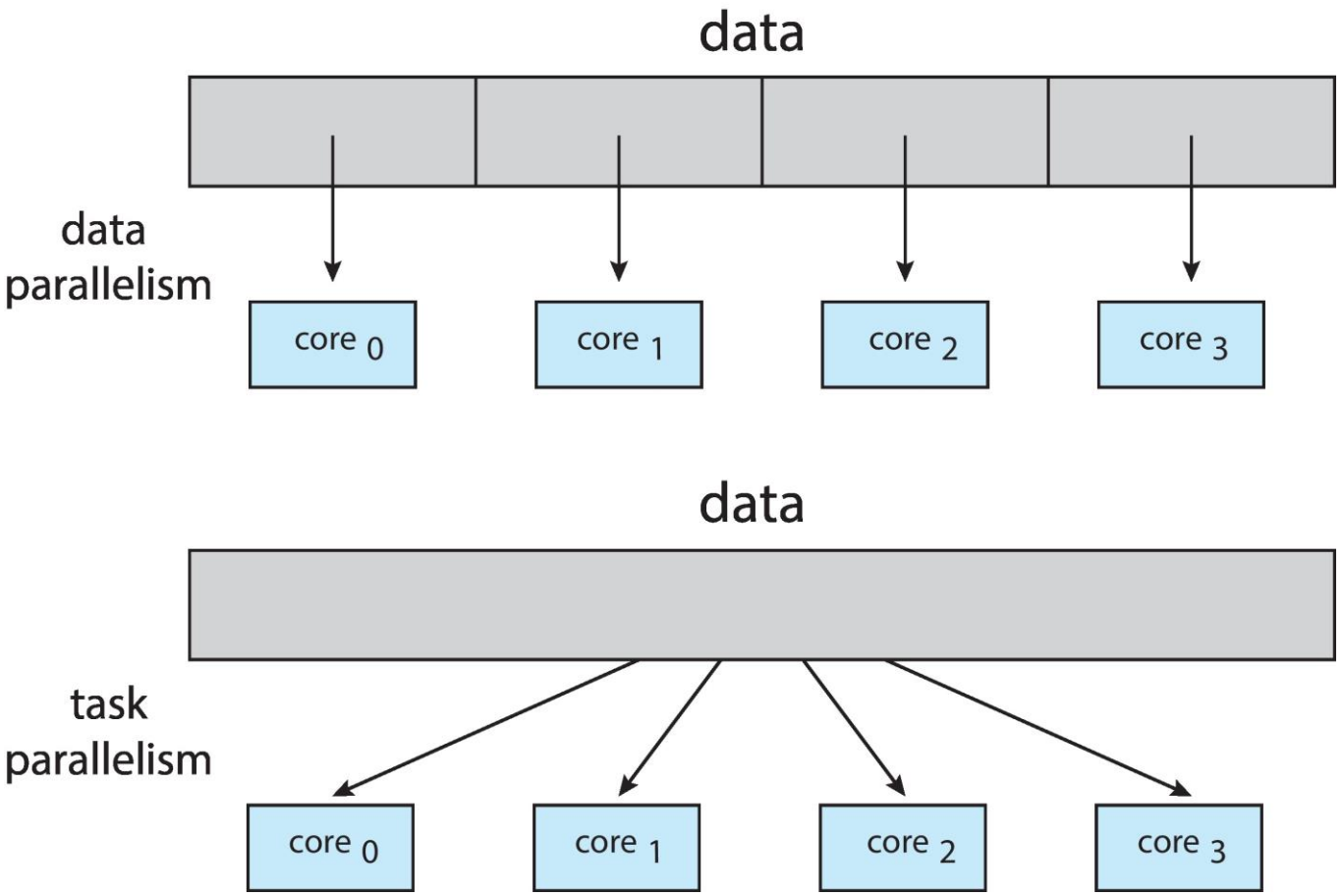
Multicore Programming

- Types of parallelism
  - **Data parallelism:** distributes subsets of the same data across multiple cores, same operation on each.
  - **Task parallelism:** distributing threads across cores, each thread performing unique operation.





Data and Task Parallelism







Amdahl's Law

- Identifies performance gains from adding additional cores to an application that has both serial and parallel components
- S is serial portion
- N processing cores

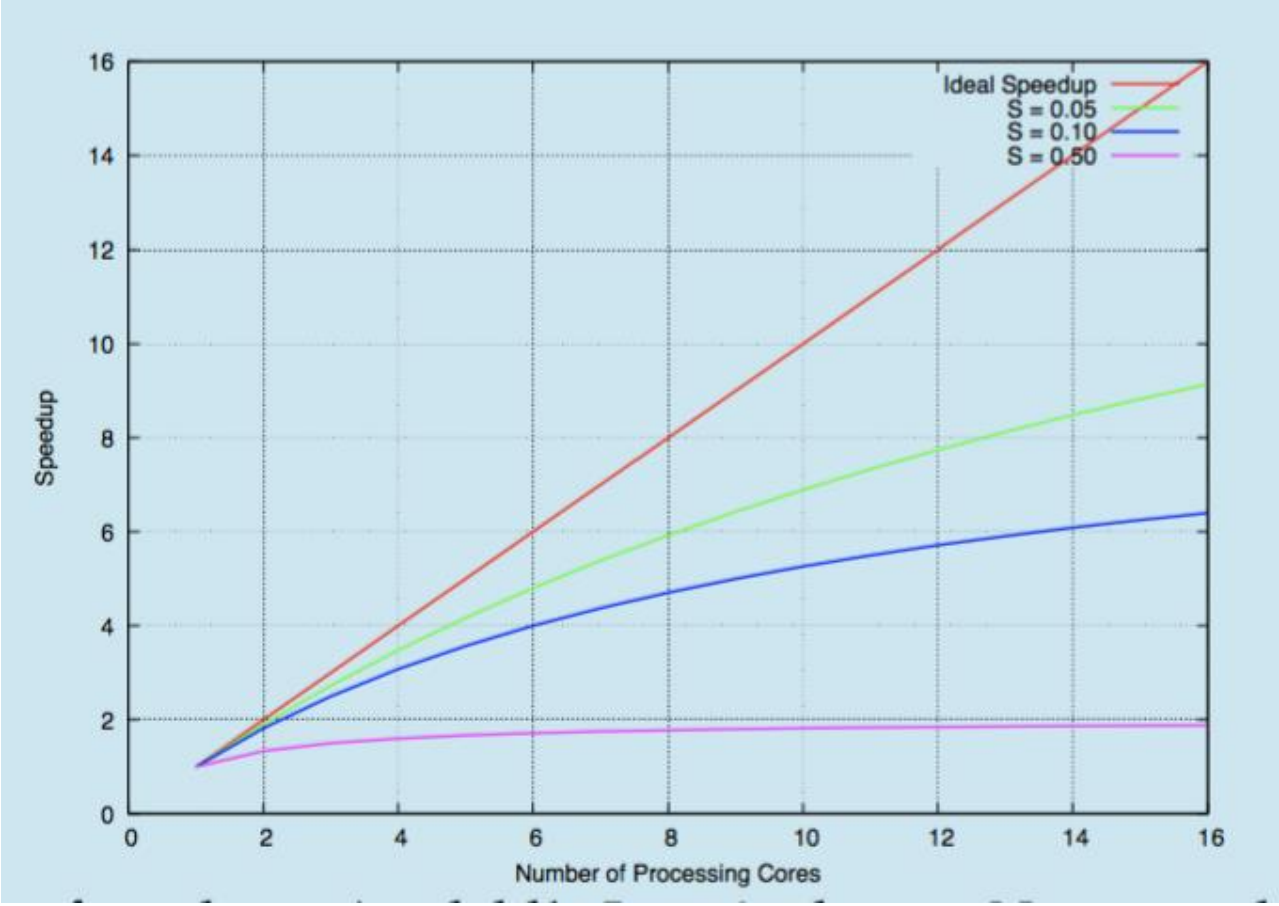
$$speedup \leq \frac{1}{S + \frac{(1-S)}{N}}$$

- That is, if application is 75% parallel / 25% serial, moving from 1 to 2 cores results in speedup of 1.6 times
- As N approaches infinity, speedup approaches 1 / S
- **Serial portion of an application has disproportionate effect on performance gained by adding additional cores**
- But does the law take into account contemporary multicore systems?





Amdahl's Law

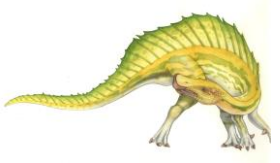

Ideal Speedup
S = 0.05
S = 0.10
S = 0.50
Speedup
Number of Processing Cores
0
2
4
6
8
10
12
14
16





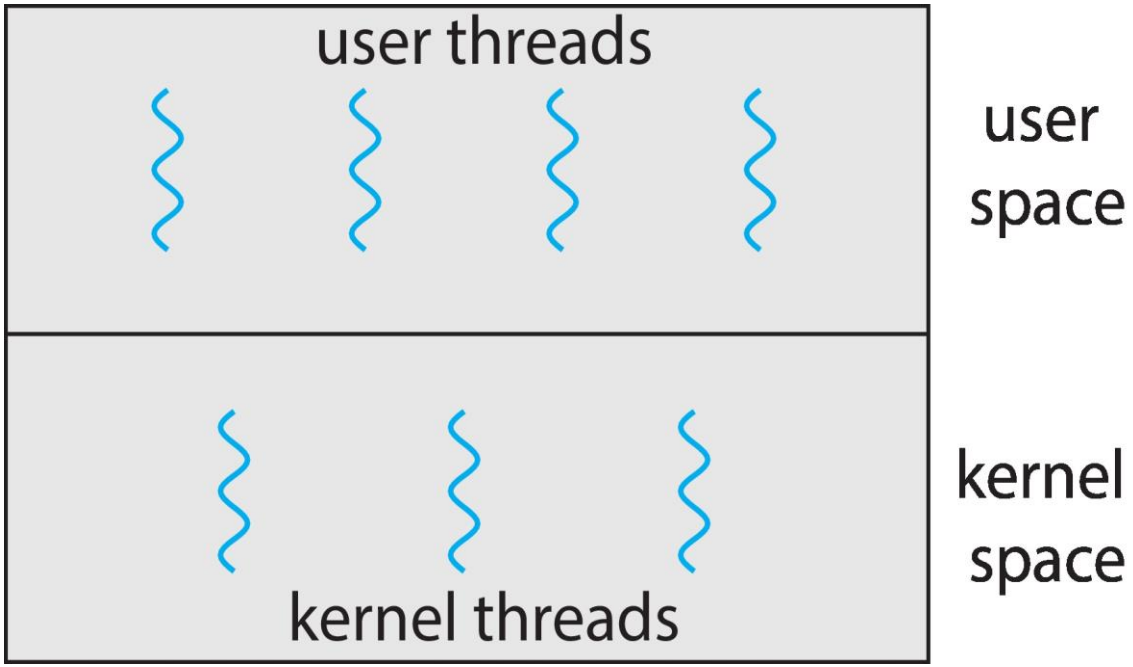
User Threads and Kernel Threads

- **User threads** - management done by user-level threads library
- Three primary thread libraries:
  - POSIX Pthreads
  - Windows threads
  - Java threads
- **Kernel threads** - Supported by the Kernel
- Examples – virtually all general -purpose operating systems, including:
  - Windows , Linux
  - Mac OS X
  - iOS, Android





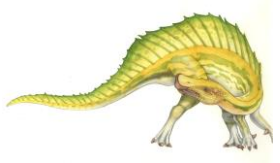
User and Kernel Threads







Multithreading Models





Multithreading Models

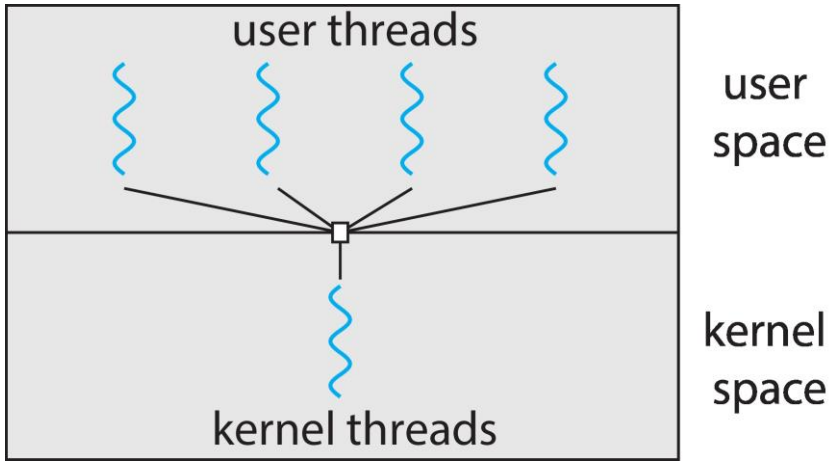
- Many-to-One
- One-to-One
- Many-to-Many





Many-to-One

- Many user-level threads mapped to single kernel thread
- One thread blocking causes all to block
- Multiple threads may not run in parallel on muticore system because only one may be in kernel at a time
- Few systems currently use this model.

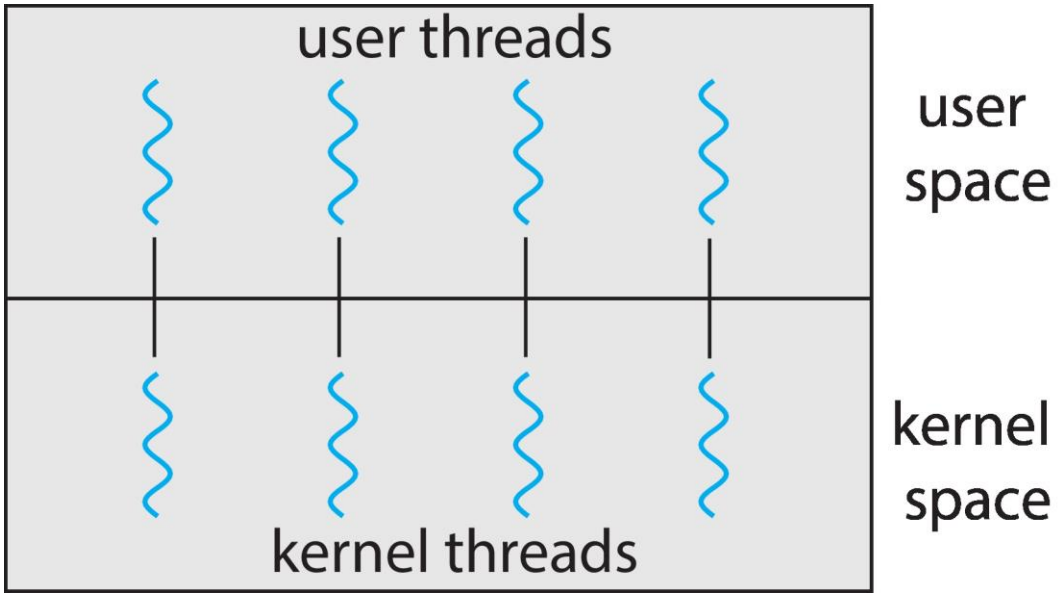






One-to-One

- Each user-level thread maps to kernel thread
- Creating a user-level thread creates a kernel thread
- More concurrency than many-to-one
- Number of threads per process sometimes restricted due to overhead
- Examples
  - Windows
  - Linux

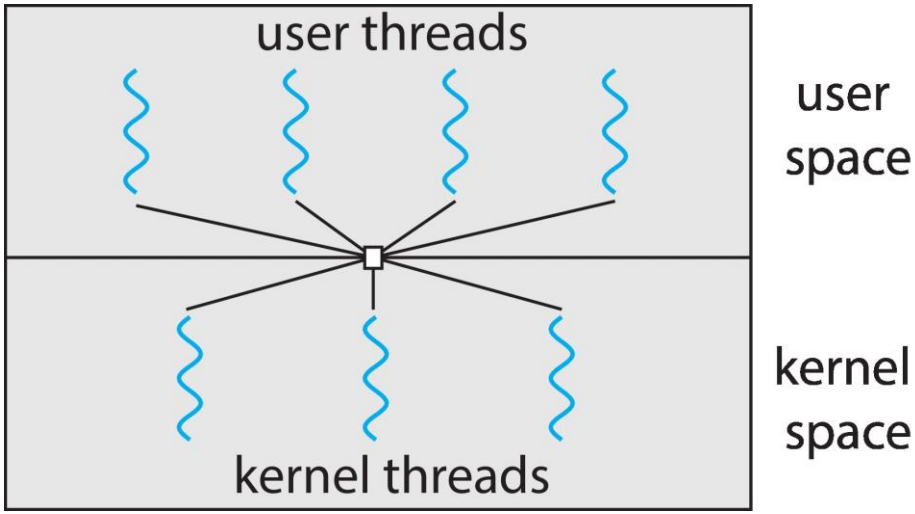






Many-to-Many Model

- Allows many user level threads to be mapped to many kernel threads
- Allows the operating system to create a sufficient number of kernel threads
- Windows with the **ThreadFiber** package
- Otherwise not very common

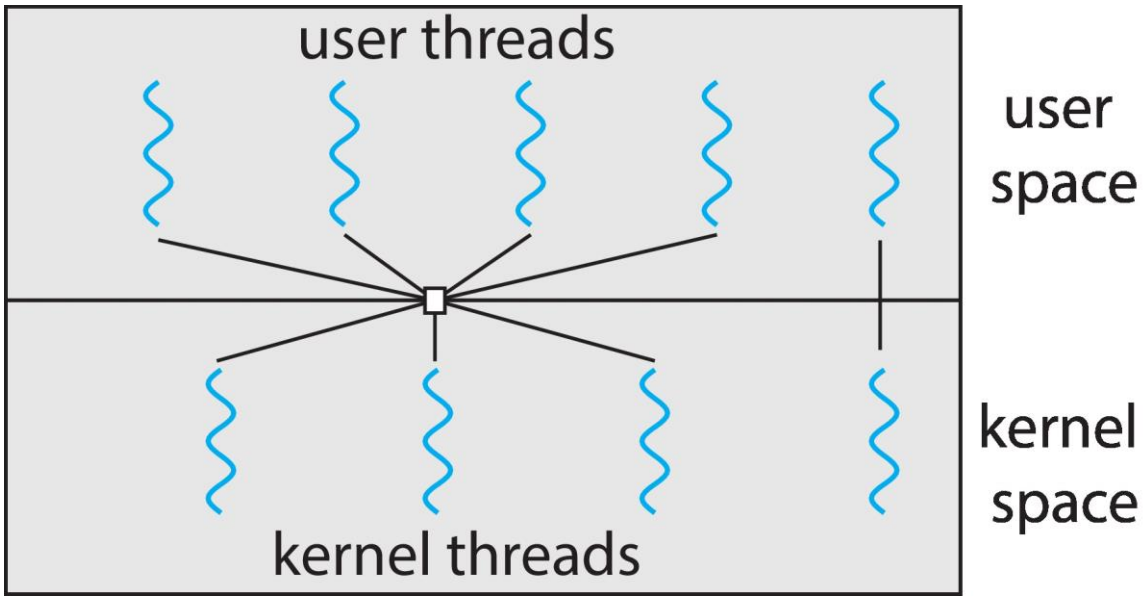


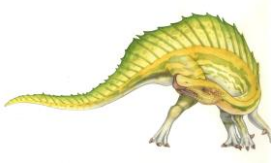




Two-level Model

- Similar to M:M, except that it allows a user thread to be bound to kernel thread.







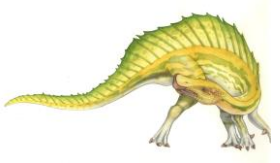
Thread Libraries





Thread Libraries

- **Thread library** provides programmer with API for creating and managing threads
- Two primary ways of implementing
  - Library entirely in user space
  - Kernel-level library supported by the OS





Pthreads

- May be provided either as user-level or kernel-level
- A POSIX standard (IEEE 1003.1c) API for thread creation and synchronization
- Specification, not implementation
- API specifies behavior of the thread library, implementation is up to development of the library
- Common in UNIX operating systems (Linux & Mac OS X)





Pthreads Example

```
#include <pthread.h>
#include <stdio.h>

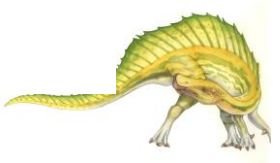
#include <stdlib.h>

int sum; /* this data is shared by the thread(s) */
void *runner(void *param); /* threads call this function */

int main(int argc, char *argv[])
{
   pthread_t tid; /* the thread identifier */
   pthread_attr_t attr; /* set of thread attributes */

   /* set the default attributes of the thread */
   pthread_attr_init(&attr);
   /* create the thread */
   pthread_create(&tid, &attr, runner, argv[1]);
   /* wait for the thread to exit */
   pthread_join(tid,NULL);

   printf("sum = %d\n",sum);
}
```



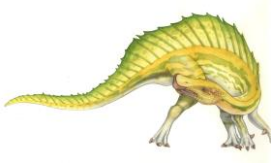


Pthreads Example (Cont.)

```
/* The thread will execute in this function */
void *runner(void *param)
{
   int i, upper = atoi(param);
   sum = 0;

   for (i = 1; i <= upper; i++)
      sum += i;

   pthread_exit(0);
}
```





Java Threads

- Java threads are managed by the JVM
- Typically implemented using the threads model provided by underlying OS
- Java threads may be created by:
  - Extending Thread class
  - Implementing the Runnable interface

```
public interface Runnable
{
    public abstract void run();
}
```

  - Standard practice is to implement Runnable interface





Java Threads

- Implementing Runnable interface:

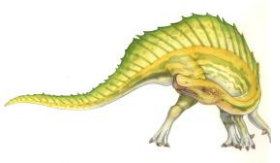
```
class Task implements Runnable
{
   public void run() {
      System.out.println("I am a thread.");
   }
}
```

- Creating a thread:

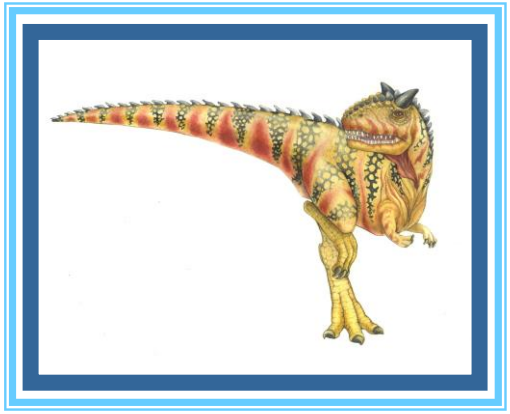
```
Thread worker = new Thread(new Task());
worker.start();
```

- Waiting on a thread:

```
try {
   worker.join();
}
catch (InterruptedException ie) { }
```



End of Chapter 4



Silberschatz, Galvin and Gagne ©2018



THANK YOU

